Similarity Search

(Part 2)

OUTLINE

1. Introduction to similarity search (Part 1)
2. Similarity search in low dimensions (Part 1)
   - kd-trees
   - Curse of dimensionality
3. Similarity search in high dimensions
   - Approximate Near Neighbor (ANN) search
   - Locality Sensitive Hashing (LSH)

Similarity search in high dimensions

r-NNS in high dimensions

- The curse of dimensionality can be tackled resorting to approximate approaches.
- How can approximation affect the solution to the r-NNS problem for a query point q and radius r?
  - The returned point p is at distance $> r$ from q (*false positive*). This is acceptable only if $d(p, q)$ is not too larger than r.
  - No r-near neighbors are returned while some exists (*false negatives*). This should be avoided.

(c, r)-Approximate Near Neighbor Search

Definition: (c, r)-Approximate Near Neighbor Search ((c, r)-ANNS)

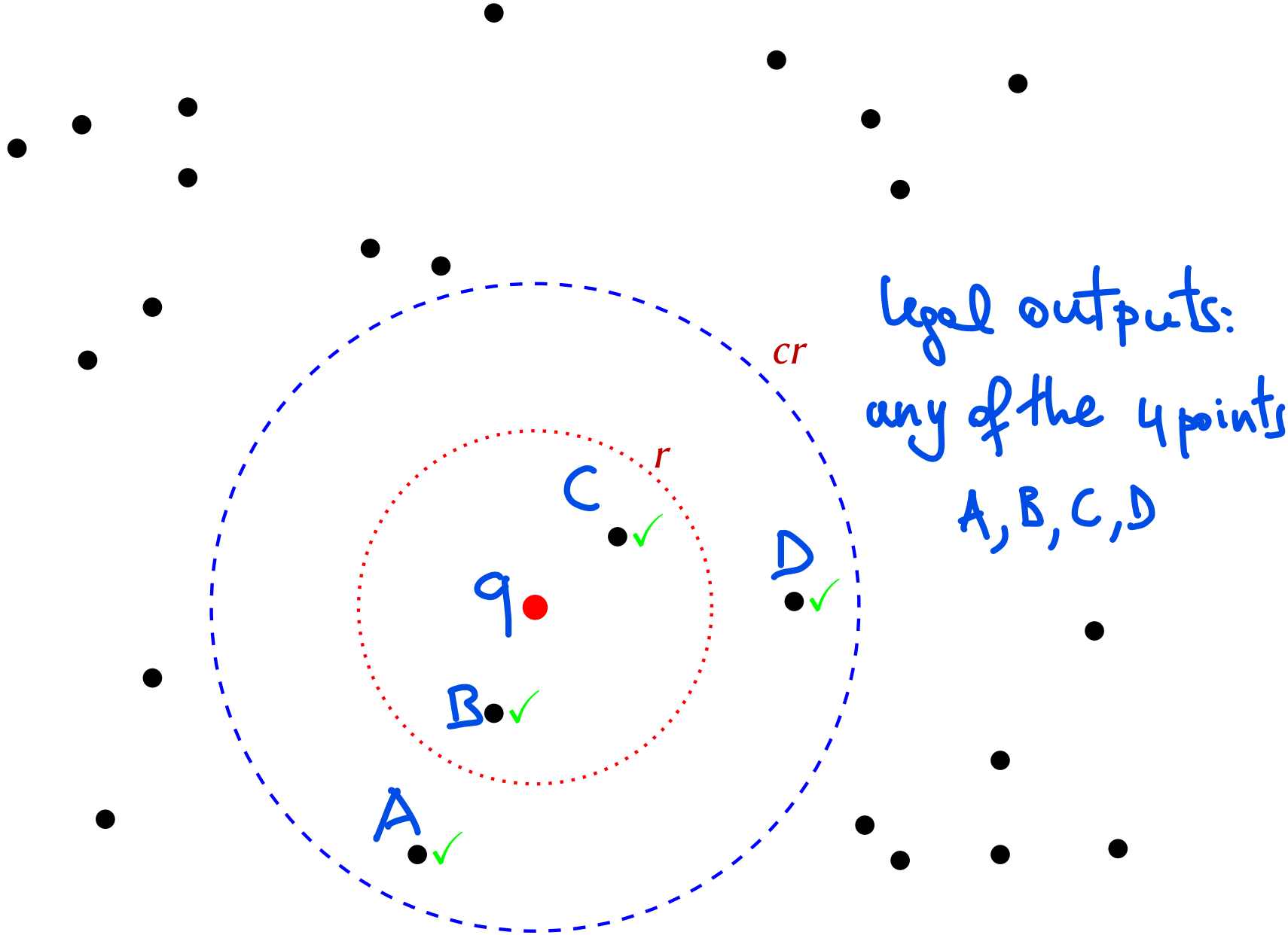
Given a set P of n points from the metric space (M, d), construct a data structure that, given a query point $q \in M$ and a distance threshold $r > 0$, provides the following answer for a certain constant $c \geq 1$:

- If there are points in $B_r(q) \cap P$, it returns a point $p \in P$ with $d(p, q) \leq cr$;
- If there are no points in $B_r(q) \cap P$ it may return either `null` or a point $p \in P$ with $d(p, q) \leq cr$, if one exists.

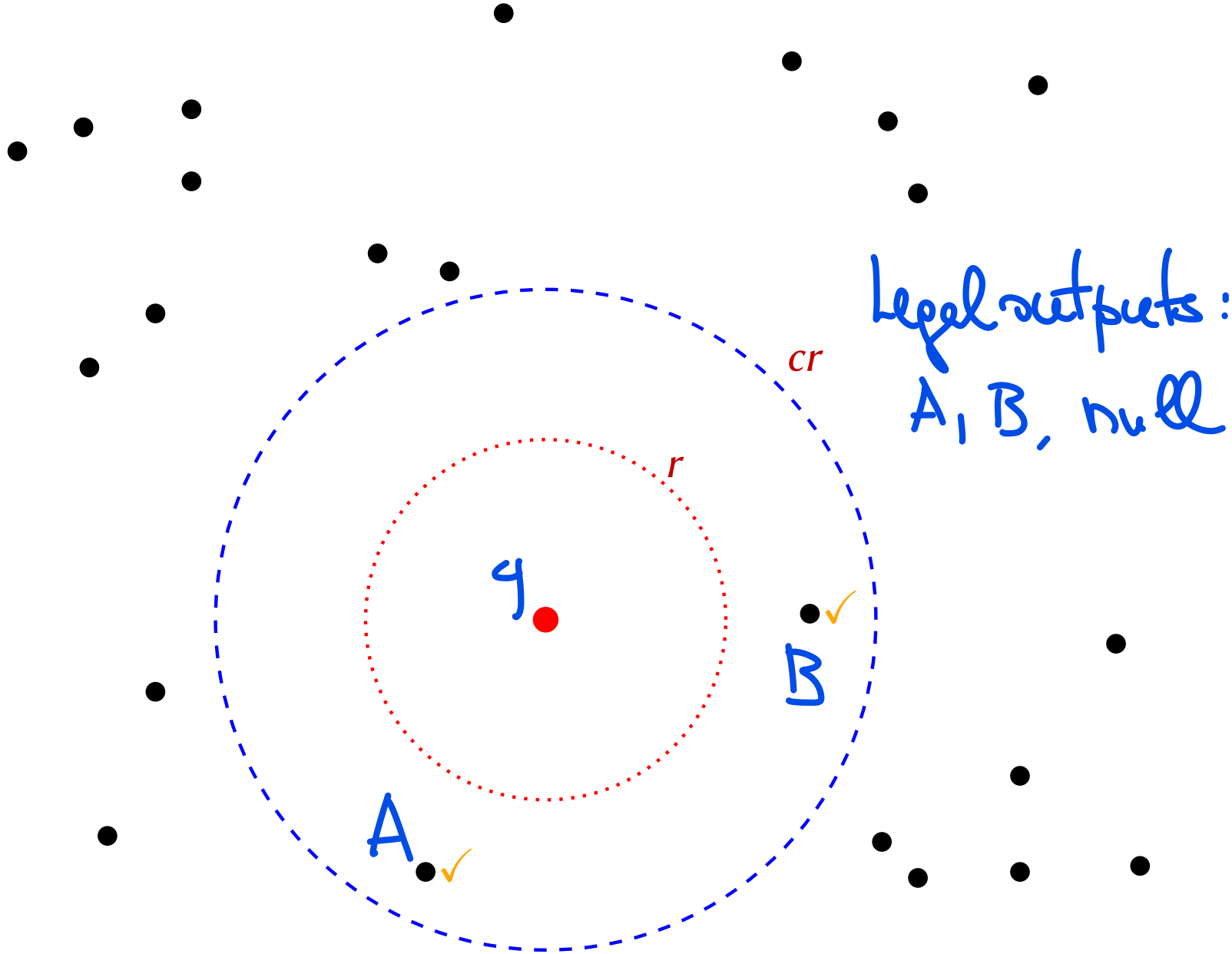
  Remark: *In this case, a* `null` *answer is always acceptable, even if a point $p \in P$ with $d(p, q) \leq cr$ exists.*

Observation: in all cases, the data structure never returns a point far away from the query point q (i.e., at distance $> cr$).

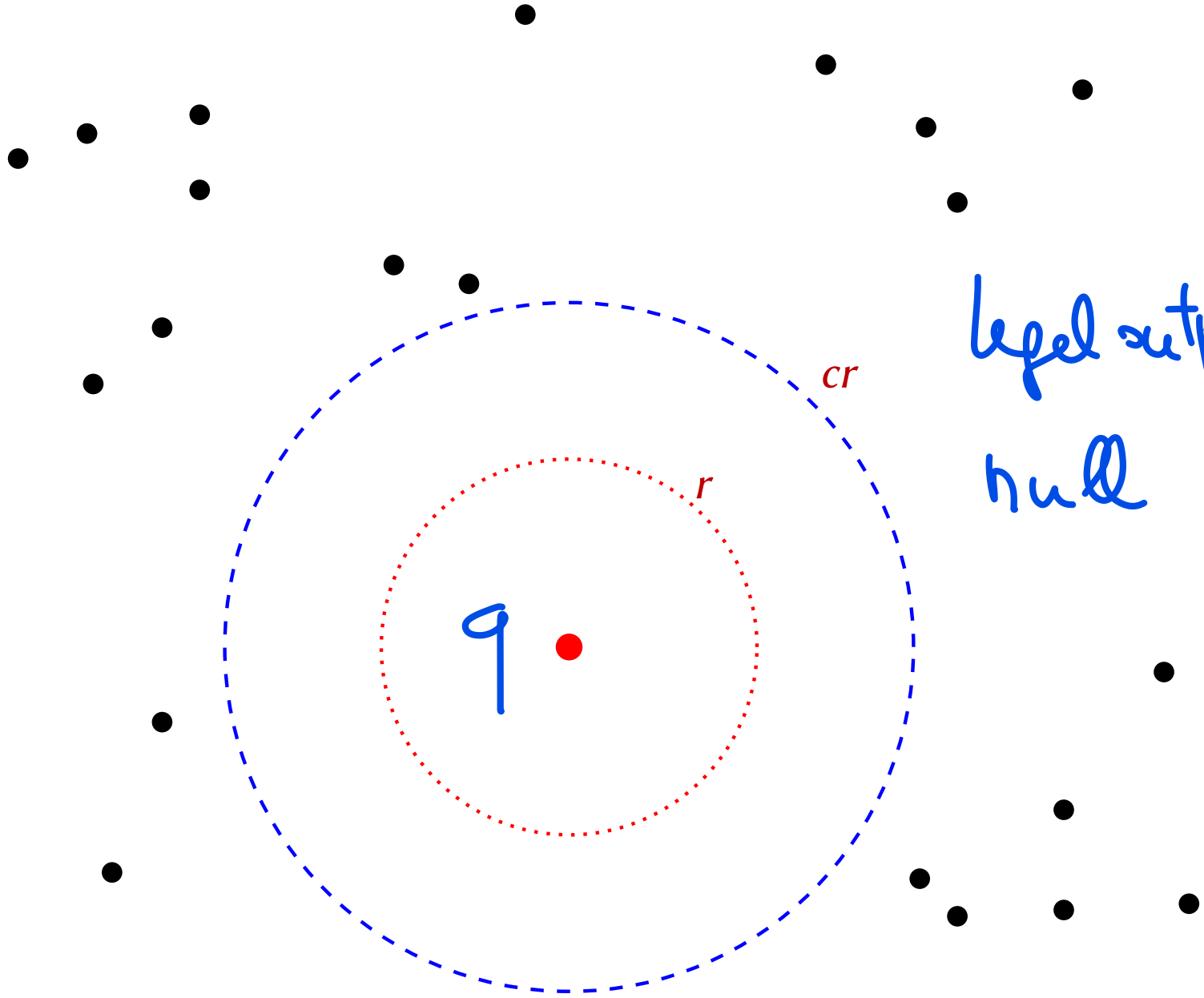
(c, r)-ANNS: example with near points



(c, r)-ANNS: example with no near points



(c, r)-ANNS: example with only far points



ANNS with Locality Sensitive Hashing

We present a solution to (c, r)-ANN based on *Locality Sensitive Hashing* (LSH), a popular technique introduced in the late 90's, which is widely used for high-dimensional near-neighbor search in several applications: e.g., recommender systems, detection of duplicates/plagiarism, search engines.

Main ideas:

- The entire space is partitioned into regions through a hash function h randomly extracted from a suitable family $\mathcal{H}$.
- The partitioning ensures that: (1) two near points are likely to be mapped by h to the same region; (2) two far points are likely to be mapped by h to different regions;
- The neighbors of the query point q are searched for in the region identified by $h(q)$.

Observation: standard hashing aims at minimizing the collision probability for any pair of elements, whereas LSH makes the collision probability positively related to the similarity between elements.

Definition of LSH

Consider a metric space (M, d) and a family of hash functions

$$\mathcal{H} = \{h : M \to S\},$$

where S is a given domain (e.g., indices in some range $[0, t]$). For any two points $p, q \in M$ denote by

$$\Pr_{h \in \mathcal{H}}[h(p) = h(q)]$$

the probability that a hash function h extracted from $\mathcal{H}$ uniformly at random maps p and q to the same value.

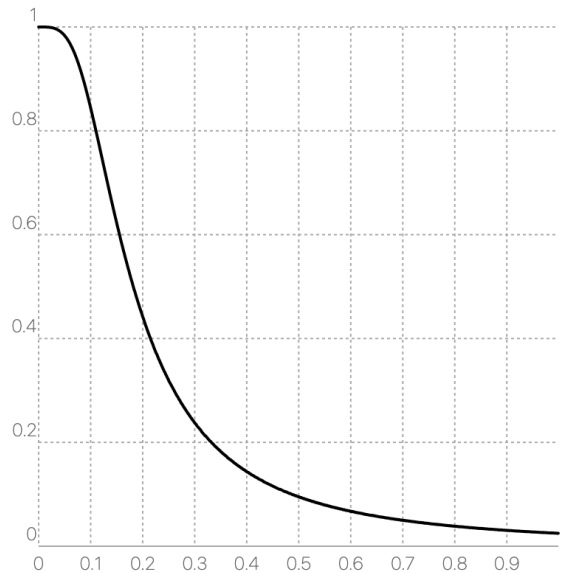
Definition: (c, r, p_1, p_2)-Locality Sensitive Hashing

Given parameters $c > 1$, $r > 0$, and $p_1, p_2 \in [0, 1]$, with $p_1 > p_2$, we say that $\mathcal{H}$ is (c, r, p_1, p_2)-locality sensitive if for any $p, q \in M$:

- If $d(p, q) \leq r$, then $\Pr_{h \in \mathcal{H}}[h(p) = h(q)] \geq p_1$.
- If $d(p, q) > cr$, then $\Pr_{h \in \mathcal{H}}[h(p) = h(q)] \leq p_2$.

Comments of LSH

- The collision probability for points with distance in $(r, cr]$ is not specified.
- Usually, the collision probability between two points p, q can be expressed as a monotonically decreasing function of $d(p, q)$.
- Example: given a pair of points with Hamming distance d (normalized in $[0, 1]$), there exists an LSH with the following collision probability (d on x-axis, collision probability on y-axis):



LSH for (c, r)-ANNS

A (c, r, p_1, p_2)-LSH $\mathcal{H}$ can be used for solving (c, r)-ANNS on the input set P, as follows:

Construction of the data structure

- Randomly select h from $\mathcal{H}$.
- Insert all points of P into a hash table T using function h

  Let $T[j]$ be the (potentially empty) bucket containing all points in P with hash value j (i.e., $T[j] = \{p \in P : h(p) = j\}$).

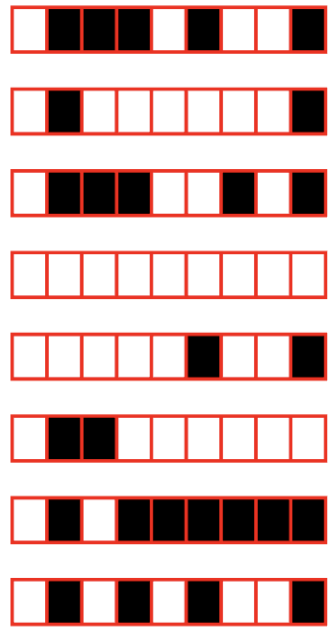
Query

- For a given query q, scan $T[h(q)]$ until a point p with $d(p, q) \leq cr$ is found, and return it. If there is no such point, return `null`.

We assume that the T[j]'s are implemented as LISTs

Example

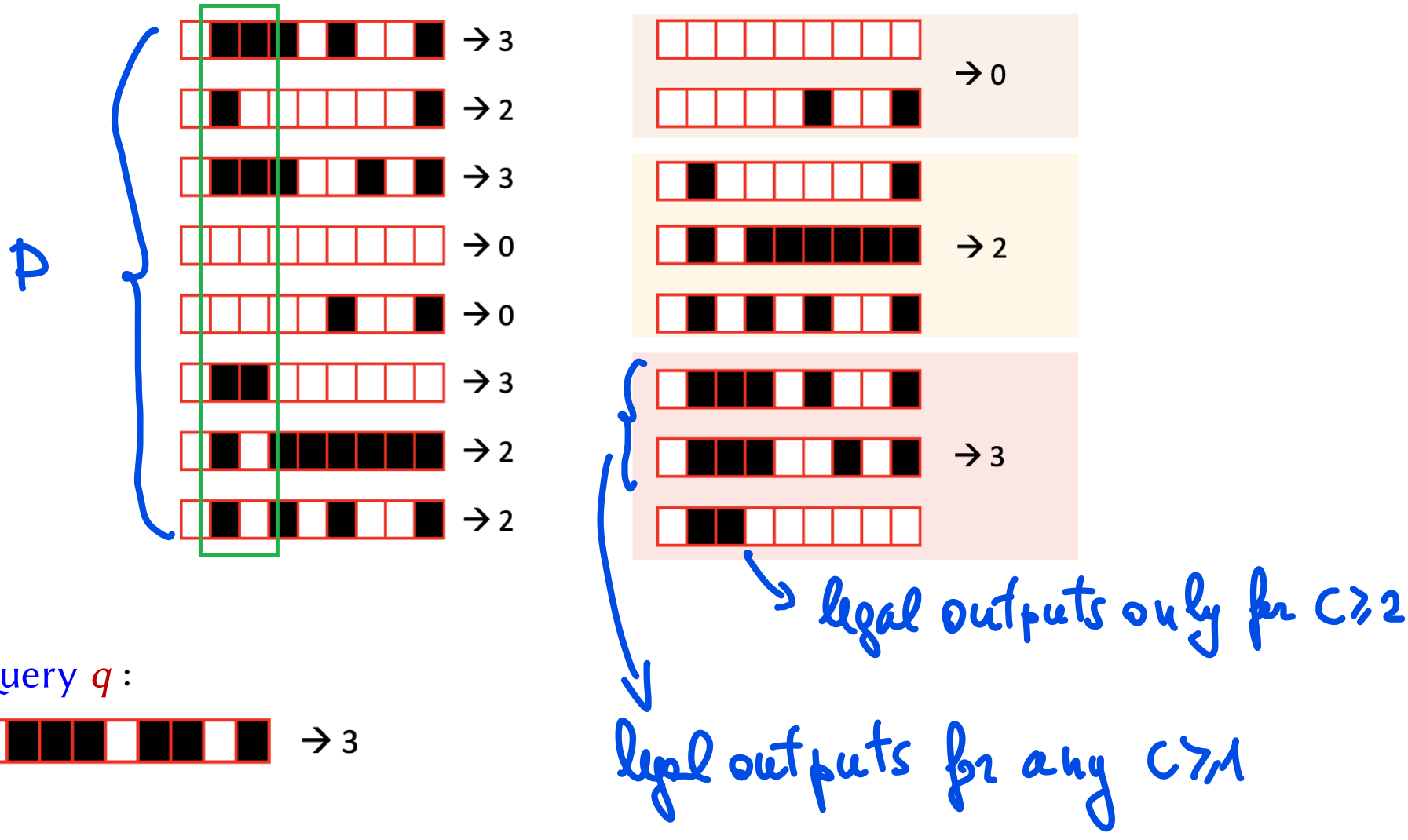
- Metric space: $M = \{\text{Boolean vectors of length } 9\}$
- Distance function: Hamming distance
- $P \subset M$ set of $n = 8$ vectors of length 9



- For $0 \leq i < j < 8$, let $h_{i,j} : M \to [0, 3]$ be the hash function mapping any vector $x \in M$ into the integer with binary configuration $x[i]x[j]$. Define:

$$\mathcal{H} = \{h_{i,.j} \ ; \ 0 \leq i < j < 8\}$$

Construction: suppose that $h_{1,2} \in \mathcal{H}$ is selected.



Query q :

ANNS with LSH

For a query point q, which points do we expect to see in $T[h(q)]$?

- A near point p (i.e., $d(p, q) \leq r$) will be in $T[h(q)]$ with probability *at least* p_1, but it might also end up in a different bucket.
- A far point p' (i.e., $d(p', q) > cr$) might end up in $T[h(q)]$, but only with probability *at most* p_2.

Worst case scenario: P contains

- 1 near point $p \in P$ (i.e., with $d(q, p) \leq r$);
- $n - 1$ far points p' (i.e., with $d(p', q) > cr$).

$\Rightarrow$ In expectation, at most np_2 far points collide with q.

ANNS with LSH: performance

We assume that (M, d) is a space of dimensionality D (e.g., $M = \mathbb{R}^D$), for some (large) D, and that:

- Each point of M requires $O(D)$ words to be stored.
- Hash values and distances can be computed in $O(D)$ time.

Theorem

Let P be a set of n points in a D-dimensional metric space (M, d), and let $\mathcal{H}$ be a (c, r, p_1, p_2)-locality sensitive family of hash functions. Using $\mathcal{H}$ and the above approach to (c, r)-ANNS, a query q is answered successfully with probability $\geq p_1$. Moreover the following performance is obtained:

- **Construction time:** $O(Dn)$
- **Space:** $O(Dn)$
- **Query time:** $O(Dnp_2)$ *in expectation.*

Proof

* PROBABILISTIC CORRECTNESS

CASE 1: $B_2(q) \cap P \neq \emptyset$ In this case, any $p' \in P$ with $d(p', q) \leq cr$ is a legal output. Consider an arbitrary point $p \in B_2(q) \cap P$ (one must exist), and let $h \in \mathcal{H}$ be the selected hash function. Then

$P_2(\text{answer is correct}) =$

$P_2(\text{answer} \neq \text{null}) \geq$

$P_2(h(p) = h(q)) \geq p_1$

Since $\mathcal{H}$ is (c, r, p_1, p_2)-locality sensitive

CASE 2: $B_r(q) \cap P = \emptyset$. In this case, any answer (either a point p with $d(p,q) \leq cr$ or null) is correct

* CONSTRUCTION TIME
* SPACE

} Straightforward

* QUERY TIME: The scan of the list $T[h(q)]$ is stopped as soon as a point p with $d(p,q) < cr$ is found, or the end of the list is reached

This implies that the query time is $O(D \cdot x)$ where x is the number of points $p \in P$ such that $d(p,q) > cr$ and $h(p) = h(q)$. Clearly, x is a random variable with $E[x] \leq n p_2$ since, by the properties of $\mathcal{H}$, for each $p \in P$ with $d(p,q) > cr$ (there are $\leq n$ such points) we have that $P_2(h(p) = h(q)) \leq p_2$

$\Rightarrow$ The expected Query time is $O(D \cdot n \cdot p_2)$

$\square$

LSH for Hamming and Euclidean distances

LSH for Hamming distance: bit sampling

- Metric space: $M = \{\text{Boolean vectors of length } D\}$
- For $0 \leq i < D$, let $h_i : M \to \{0, 1\}$ be the hash function that maps each vector $x \in M$ into its i-th bit $x[i]$. Define:

$$\mathcal{H}_H = \{h_i \ : \ 0 \leq i < D\}.$$

- Given two points p, q, the probability of collision is

$$\Pr_{h \in \mathcal{H}_H}[h(p) = h(q)] = 1 - \frac{d_H(p, q)}{D},$$

where $d_H()$ denotes the Hamming distance.

probability that a hash function h_i is extracted with $p[i] = q[i]$

LSH for Hamming distance: bit sampling

For any two points p, q we have that:

- If $d_H(p,q) \leq r$, then

$$\Pr_{h \in \mathcal{H}_H}[h(p) = h(q)] = 1 - \frac{d_H(p,q)}{D} \geq 1 - \frac{r}{D} \stackrel{def}{=} p_1$$

- If $d_H(p,q) > cr$, then

$$\Pr_{h \in \mathcal{H}_H}[h(p) = h(q)] = 1 - \frac{d_H(p,q)}{D} < 1 - \frac{cr}{D} \stackrel{def}{=} p_2$$

Therefore $\mathcal{H}_H$ is $(c, r, 1 - r/D, 1 - cr/D)$-locality sensitive.

p_1 p_2

Observation

A (c, r, p_1, p_2)-locality sensitive family $\mathcal{H}$ of hash functions if effective when $p_1 \gg p_2$.

A parameter which is typically used to measure the quality of such a family is the ρ factor defined as:

$$\rho = \frac{\log_2 p_1}{\log_2 p_2} = \frac{\log_2(1/p_1)}{\log_2(1/p_2)}$$

Note that since $1 \geq p_1 > p_2 > 0$, we have $\rho \in (0, 1)$, decreasing with p_1/p_2, hence the smaller ρ the better!

The ρ factor for bit sampling is:

$$\rho = \frac{\log_2 p_1}{\log_2 p_2} = \frac{\log_2(1 - r/D)}{\log_2(1 - cr/D)} \sim \frac{r/D}{cr/D} = 1/c.$$

LSH for Euclidean distance: random projection

- Metric space: $M = \mathbb{R}^D$
- For a fixed value $w > 0$ and parameters $a \in \mathbb{R}^D$ and $b \in [0, w]$, define the following hash function $h_{a,b}(p) : M \to \mathbb{Z}$:

$$h_{a,b}(p) = \left\lceil \frac{\langle a, p \rangle + b}{w} \right\rceil, \quad (\langle , \rangle \text{ denotes the inner product})$$

and define

$$\mathcal{H}_E(w) = \{h_{a,b}(p) \ : \ a \in \mathbb{R}^D \text{ and } b \in [0, w]\}$$

It can be shown that $\mathcal{H}_E(w)$ is (c, r, p_1, p_2)-locality sensitive with $\rho = O(1/c)$, assuming a selected with normal distribution ($\mathcal{N}^D(0, 1)$), and b with uniform distribution

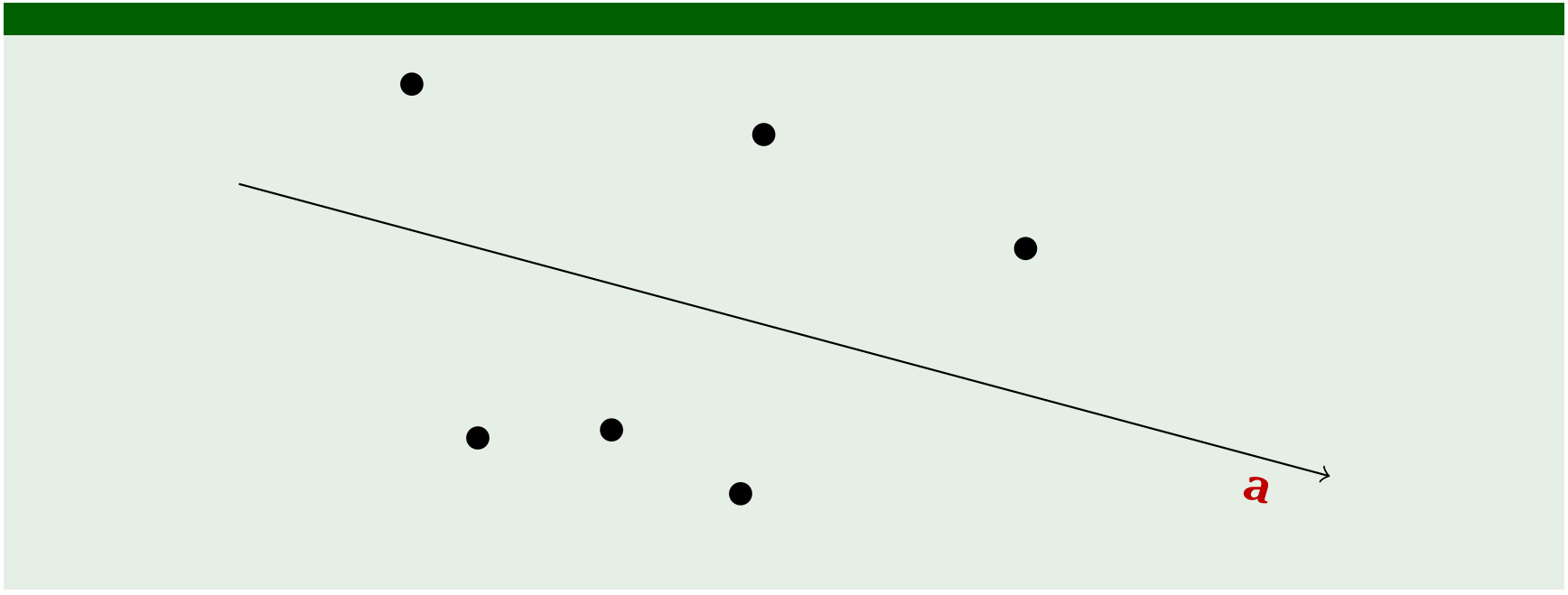
OBSERVATIONS

* The hash functions of $\mathcal{H}_E(w)$ map points $p \in \mathbb{R}^D$ onto arbitrary integers. For practical purposes, once a function $h_{a,b}$ is extracted from $\mathcal{H}_E(w)$ a further hash function h is used to map the non-empty buckets created by $h_{a,b}$ to indices in a small range. Then, for a query q the bucket $T[h_{a,b}(q)]$ is first retrieved among those mapped by h to the same index, and then

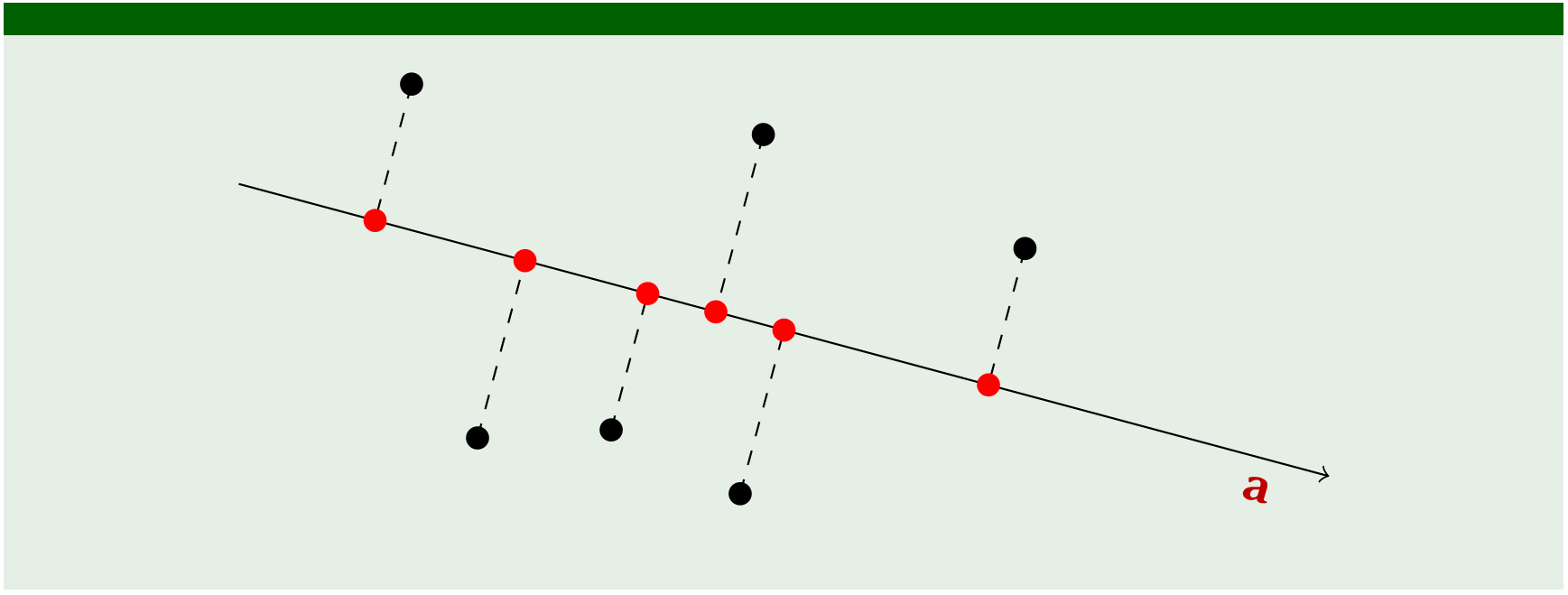
this bucket is searched for a near neighbor of q

* For Euclidean spaces, better families of locality-sensitive hash functions exist with $\rho \in O(1/c^2)$

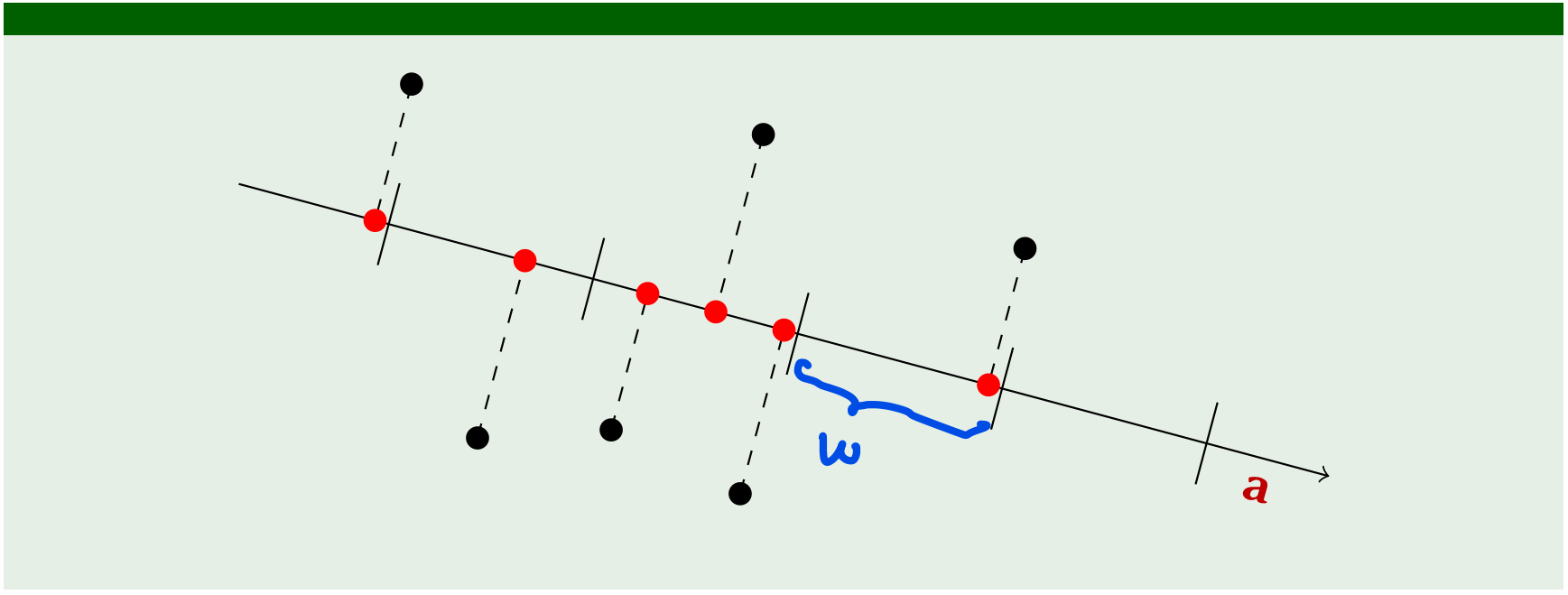
Random projection: example



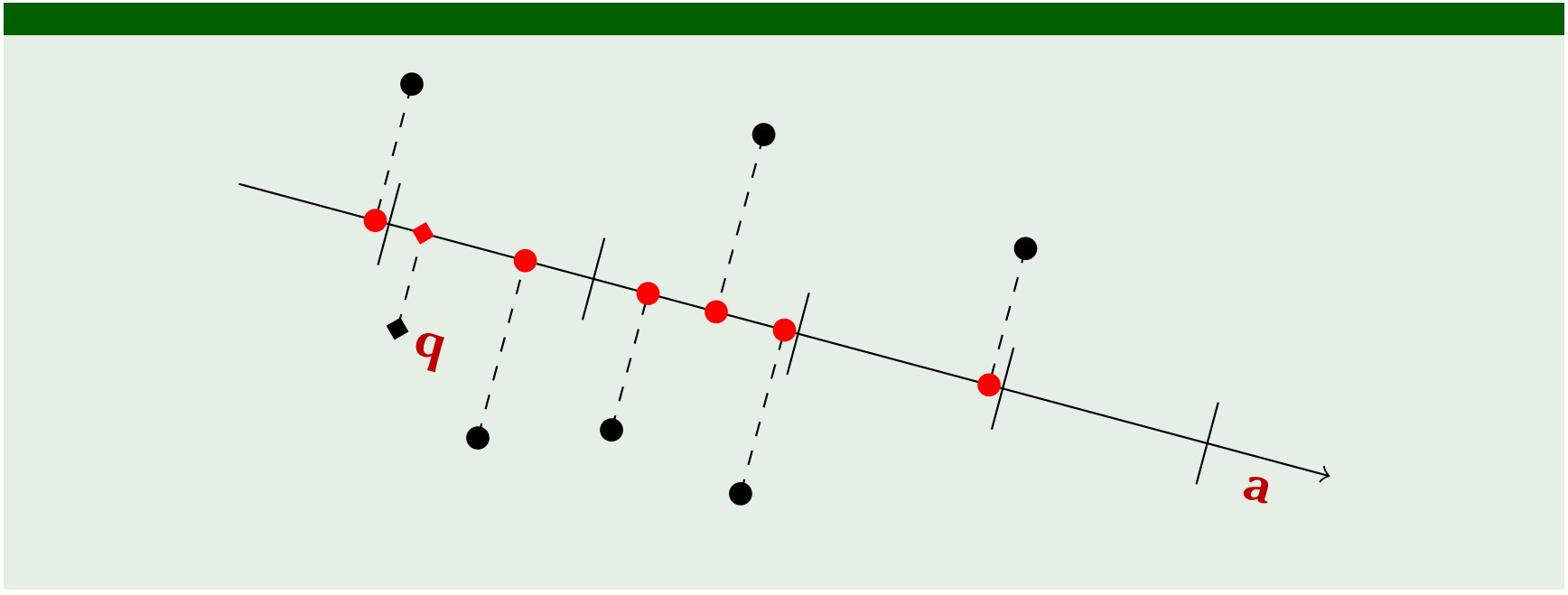
Random projection: example



Random projection: example



Random projection: example



Improving the data structure

Ideally, LSH should have p_1 close to 1 and p_2 close to 0. However, a family $\mathcal{H}$ might not provide this type of guarantees.

We now outline two improvements to respectively increase the collision probability of near points and decrease the collision probability of far points, which, combined together, yield better LSH.

Improvement 1: increase collision probability of near points

Idea: repeat search $\ell > 1$ times using ℓ distinct hash tables based on independent hash functions chosen uniformly at random from a (c, r, p_1, p_2)-locality sensitive family $\mathcal{H}$.

This technique is called repetition or OR construction.

- The probability that a given near point p (i.e., with $d(p, q) \leq r$) collides with the query point q in at least one hash table increases with ℓ (see next slides).
- However, checking ℓ buckets, one for each hash table, becomes computationally expensive.

Let p be such that $d(p,q) \leq r$

* The probability that in one particular hash table p and q are in different buckets is $\leq 1 - p_1$
* The probability that in all of the l hash tables p and q are in different buckets is $\leq (1-p_1)^l$ by independence of the hash functions

$\Rightarrow$ The probability that given the query q p is found in the same bucket as q in "at least" one hash table is

$\geq 1-(1-p)^l$

Note that for $l>1$ $\quad (1-p_1)^l < 1-p_1$

hence, $1-(1-p_1)^l > 1-(1-p_1) = p_1$

Improvement 2: decrease collision probability of far points

Idea: use a family $\mathcal{G}$ of hash functions obtained by concatenating $k \geq 1$ independent hash functions chosen uniformly at random from a (c, r, p_1, p_2)-locality sensitive family $\mathcal{H}$. Namely,

$$\mathcal{G} = \left\{ g \in \mathcal{H}^k \right\} = \{ g(p) = (h_1(p), \ldots, h_k(p)), \text{ with } h_i \in \mathcal{H} \}$$

It is immediate to establish that for a random $g \in \mathcal{G}$ and any two points p, q with $p \neq q$,

- If $d(p, q) \leq r$, then $\Pr[g(p) = g(q)] \geq p_1^k$;
- If $d(p, q) > cr$, then $\Pr[g(p) = g(q)] \leq p_2^k$.

This technique is called concatenation or AND construction.

Improvement 2: decrease collision probability of far points

Suppose that we set

$$k = \log_{1/p_2} n = \frac{\log_2 n}{\log_2(1/p_2)}.$$

Then, for a random $g \in \mathcal{G}$ and any two points p, q with $p \neq q$,

- If $d(p, q) \leq r$, then $\Pr[g(p) = g(q)] \geq p_1^k = 1/n^\rho$;
- If $d(p, q) > cr$, then $\Pr[g(p) = g(q)] \leq p_2^k = 1/n$.

  Remark: With the above choice of k, the expected number of collisions of a query point q with far points in the same bucket is at most 1.

$$k = \frac{\log_2 n}{\log_2 (1/p_2)}$$

$$= \frac{\log_2 n}{\log_2 (1/p_1)} \cdot \frac{\log_2 (1/p_1)}{\log_2 (1/p_2)} = \left(\log_{1/p_1} n\right) \cdot c$$

$$\Rightarrow p_1^k = n^c$$

LSH and (c, r)-ANNS: a general schema

We merge the two improvements to get the following schema. Let $\mathcal{H}$ be a (c, r, p_1, p_2)-locality sensitive family of hash functions, and let k and ℓ be two values that will be set later.

Construction of the data structure

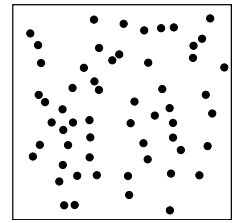
- Construct ℓ hash functions $g_1 \dots g_\ell$: each g_i consist of the concatenation of k hash functions randomly and independently selected from $\mathcal{H}$.
- For each g_i, construct a hash table T_i of points in P using g_i. We let $T_i[j]$ be the (potentially empty) bucket containing all points in P with hash value j when using g_i.

Query

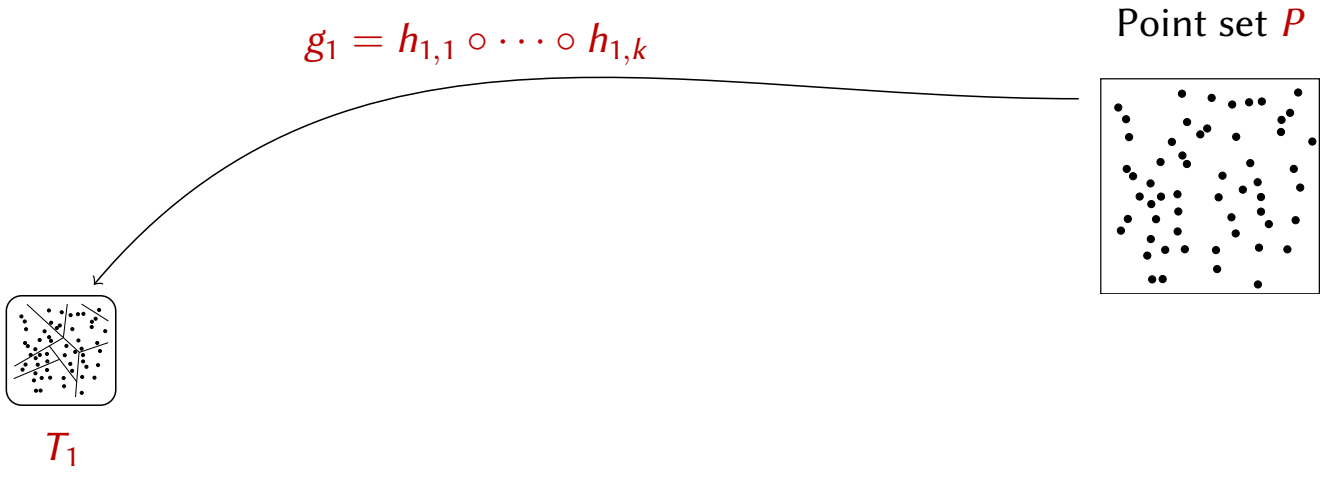
- Scan $T_1[g_1(q)], \dots T_\ell[g_\ell(q)]$ until a cr-near point p is found and return it; if no such point is found, return `null`.

LSH and (c, r)-ANNS: example

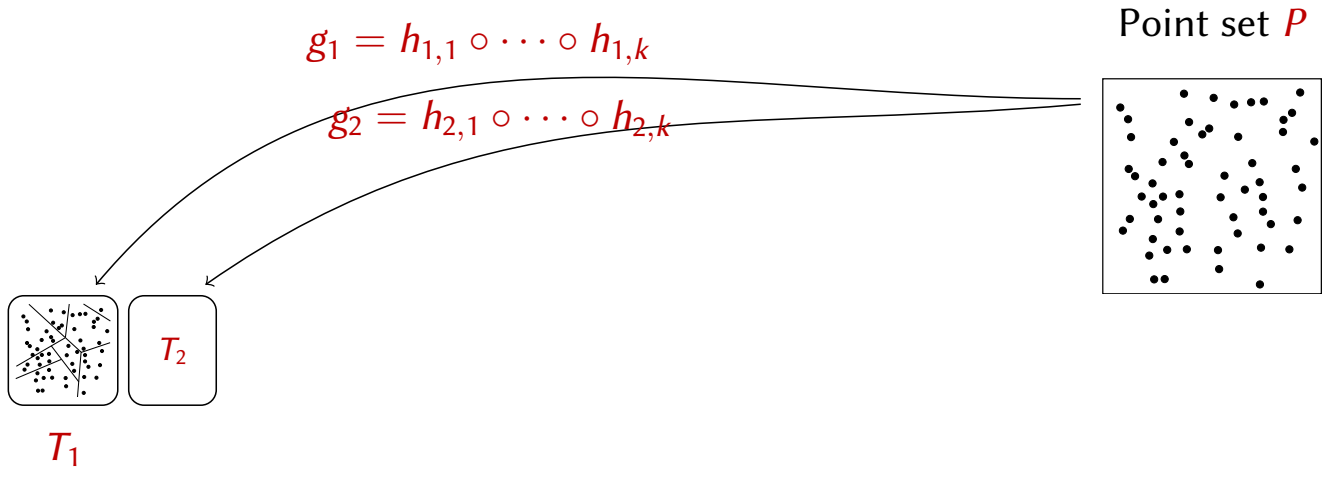
Point set P



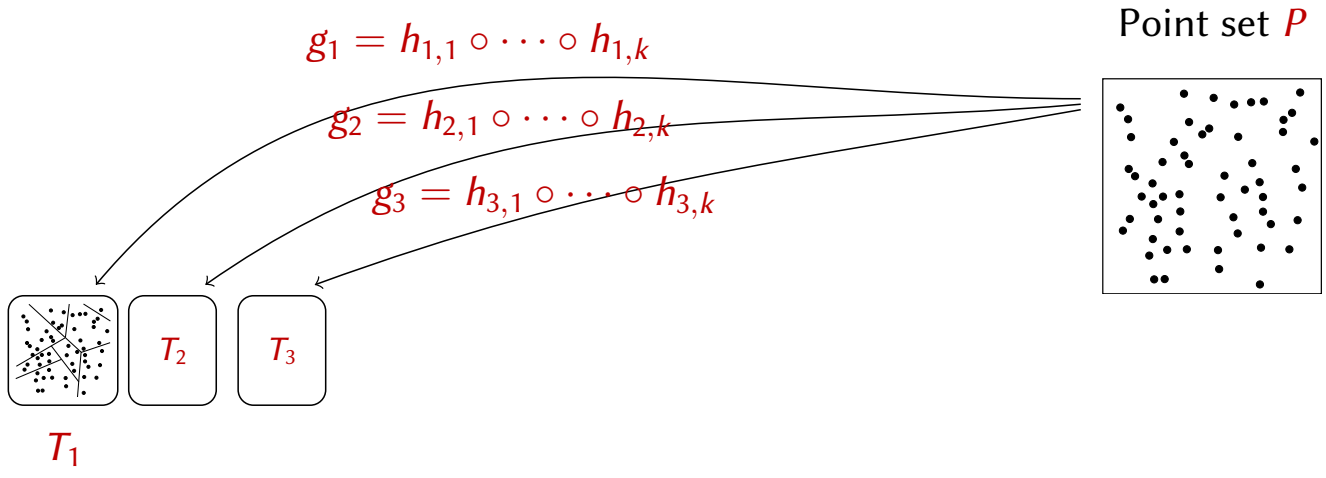
LSH and (c, r)-ANNS: example



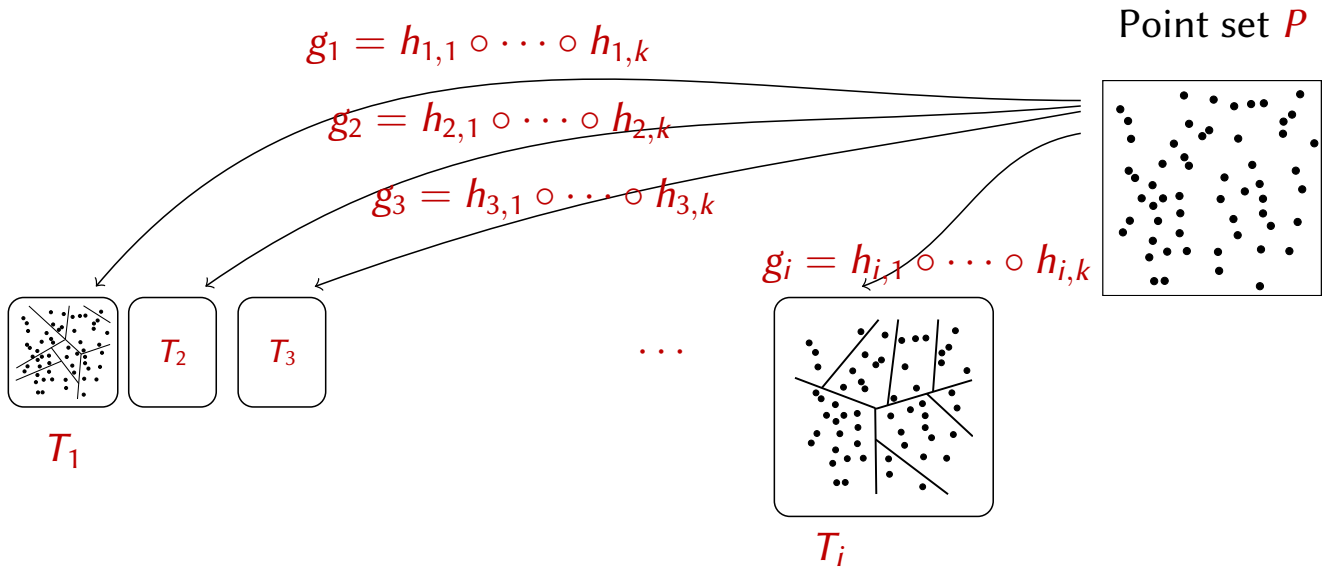
LSH and (c, r)-ANNS: example



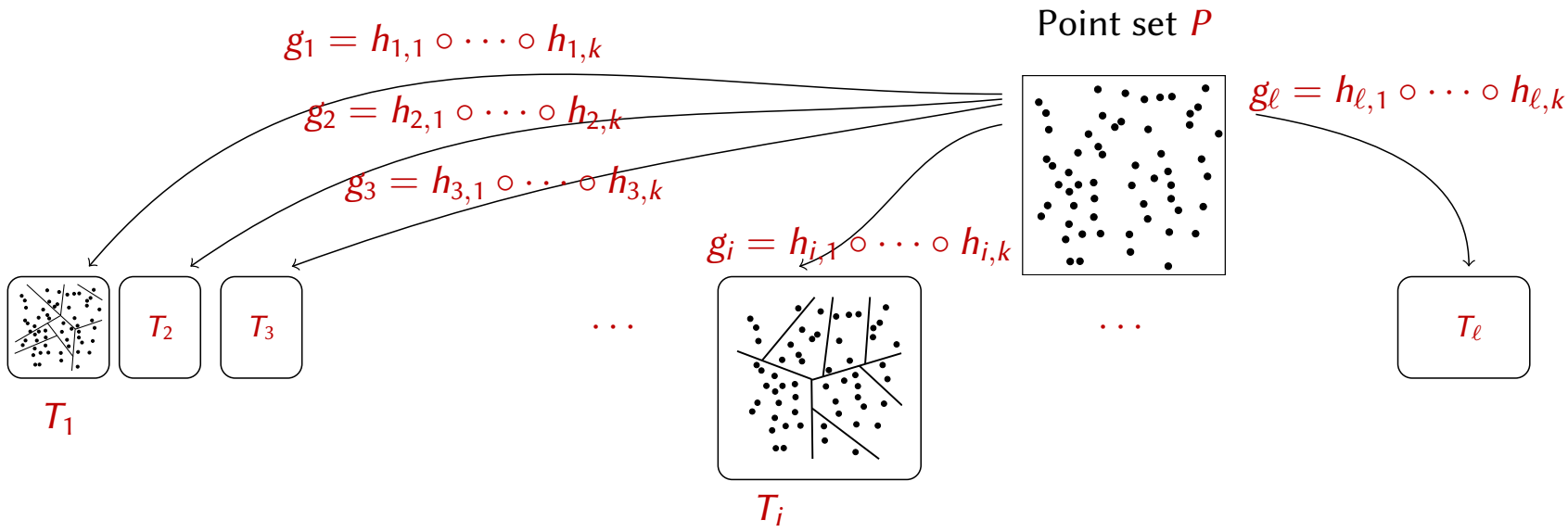
LSH and (c, r)-ANNS: example



LSH and (c, r)-ANNS: example



LSH and (c, r)-ANNS: example



LSH and (c, r)-ANNS: performance

Theorem

Let P be a set of n points in a metric space (M, d), and let $\mathcal{H}$ be a $(c, r, p_1, p_2,)$-locality sensitive family of hash functions. Fix

$$\begin{aligned} k &= \log_{1/p_2} n \quad \textit{and} \\ \ell &= 2p_1^{-k} = 2n^{\rho}, \end{aligned}$$

where $\rho = \log_2 p_1 / \log_2 p_2$. Using the above approach to (c, r)-ANNS, a query is answered successfully with probability $\geq 1/2$. Moreover the following performance is obtained:

- **Construction time:** $O\left(Dn^{1+\rho} \log_{1/p_2} n\right)$
- **Space:** $O\left(Dn + n^{1+\rho} \log_{1/p_2} n\right)$
- **Query time:** $O\left(Dn^{\rho} \log_{1/p_2} n\right)$ *in expectation.*

LSH for Hamming distance: concatenating bit sampling

Concatenating k bit sampling LSH consists in randomly selecting k indexes, and yields the following collision probabilities:

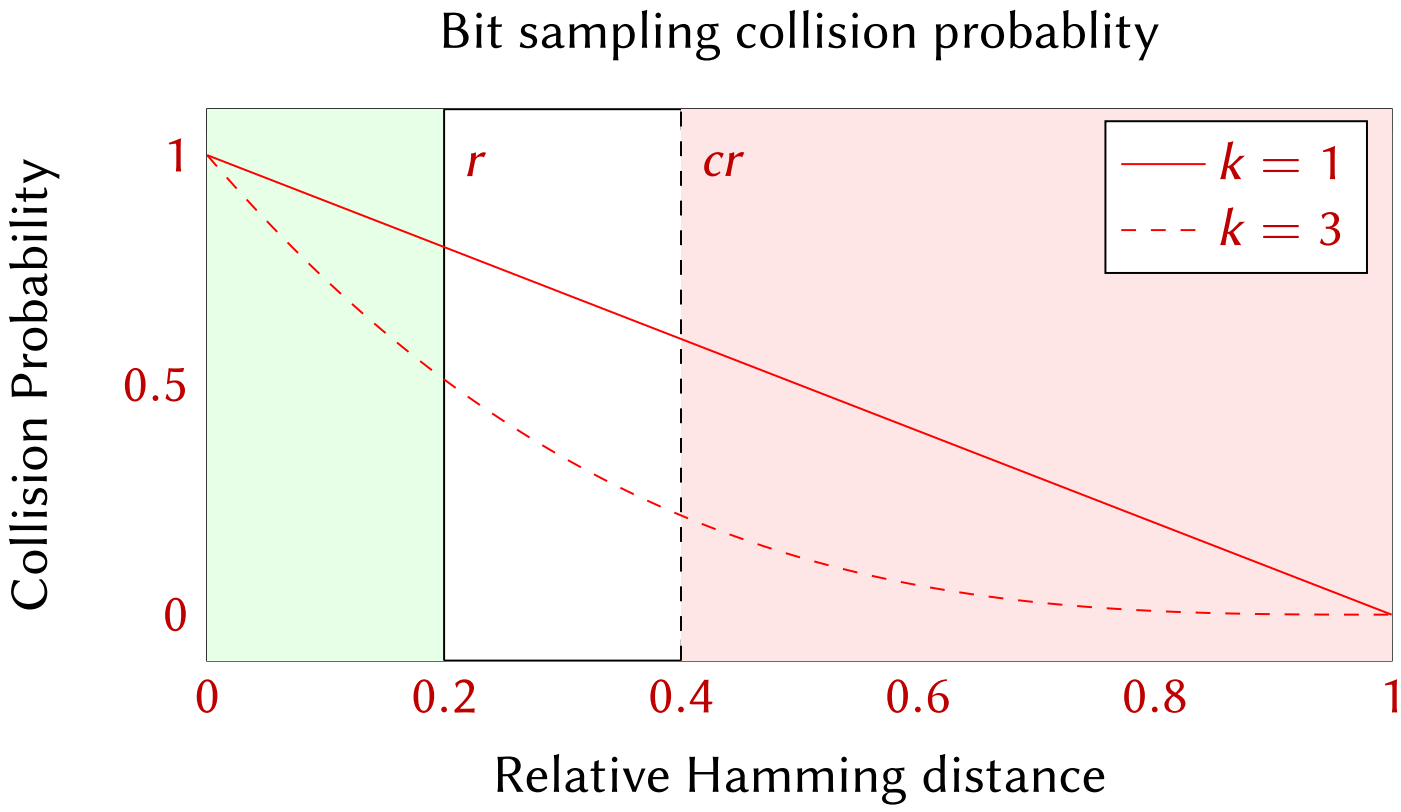
- $\Pr_{h \in \mathcal{H}_H}[h(p) = h(q)] \geq (1 - r/D)^k$, if $d_H(p, q) \leq r$;
- $\Pr_{h \in \mathcal{H}_H}[h(p) = h(q)] \leq (1 - cr/D)^k$, if $d_H(p, q) \geq cr$;

The concatenation does not change the ρ value.

Bitsampling: Project to random subset of dimensions.

x	001010010 10	y	10 10 11000 1 0
$h(x)$	011	$h(y)$	011

LSH for Hamming distance: concatenating bit sampling



Exercises

Exercise

Let P be a collection of n documents that you want to store into a suitable data structure so to retrieve, given a query document q, a similar document in P. Let W be a set of D *relevant words* and suppose that the similarity between two documents depends on the number of words of W in common (ignoring their relative frequencies).

1. Describe a representation of the documents and a data structure for P based on a suitable locality-sensitive family of hash functions (use only one hash function for the data structure).
2. Based on the above point, find values of c and r such that the (c, r)-ANNS problem for P can be solved correctly with probability at least $1/2$ and expected query time $O(n)$. (Observe that the trivial exact approach requires $O(Dn)$ query time.)

Exercise

Let P be a set of n D-dimensional Boolean vectors. Suppose that P is stored into a hash table T built using a hash function h randomly drawn from the bit-sampling LSH family $\mathcal{H} = \{h_i : 0 \leq i < D\}$. For $h \in \mathcal{H}$, let "$\mathtt{not}(h(x))$" denote the negation of the binary value $h(x)$.

1. Given two vectors p and q, determine the probability $\Pr_{h \in \mathcal{H}}[h(p) = \mathtt{not}(h(q))]$ as a function of the Hamming distance $d_H(p, q)$.
2. Given a query vector q, we want to find an *r-far* vector $p \in P$, i.e., such that $d_H(p, q) \geq r$. Based on the above analysis, how would you efficiently search such a p in the table T? What probabilistic guarantees does your method provide?

Summary

- Similarity search: r-NNS and RR problems.
- k-d tree for similarity search in low dimensions.
- Curse of dimensionality for similarity search.
- (c, r)-ANNS problem.
- LSH approach to the (c, r)-ANNS problem.
  - Definition of $(c, r, p_1, p_2))$-locality sensitive hash functions.
  - Solving (c, r)-ANNS through (c, r, p_1, p_2)-locality sensitive hash functions.
  - $(c, r, p_1, p_2))$-locality sensitive hash functions for Hamming and Euclidean distances (bit sampling and random projection).
  - Improving collision probabilities with repetition and concatenation, and their combination.

References

LRU14 J. Leskovec, A. Rajaraman and J. Ullman. Mining Massive Datasets. Cambridge University Press, 2014. Section 3.6.

BCKO08 Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. Computational Geometry: Algorithms and Applications (3rd ed. ed.). Springer-Verlag, 2008. Section 5.2

AI08 Alexandr Andoni and Piotr Indyk. 2008. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. Commun. ACM 51, 1 , 2008.